

Analytic Per-Node Aggregation Depth for Heterophilous Graph Neural Networks

Anonymous submission

Abstract

Graph Neural Networks (GNNs) typically employ a uniform neighborhood aggregation depth across all nodes. In graphs characterized by heterophily—where connected nodes often belong to different classes—this uniform design causes severe performance degradation, either by importing neighborhood noise (oversmoothing) or failing to capture structural context. Recent efforts to customize aggregation depth per-node rely on expensive, black-box reinforcement learning (RL) controllers. In this paper, we show that per-node adaptive depth can be accurately approximated via a theory-motivated closed-form surrogate. Under a degree-corrected Contextual Stochastic Block Model (DC-CSBM), we analyze the local signal-to-noise ratio (SNR) of neighborhood features as a function of propagation depth, showing how the optimal depth scales with local homophily and degree. Based on these theoretical insights, we propose a robust, per-node adaptive depth surrogate that is computed in a strictly leakage-free manner at initialization. Leveraging this surrogate, we introduce **AGT-Implicit**, an RL-free architecture that replaces trial-and-error controllers with our theoretically motivated formula to compute the explicit neighborhood view. To capture distant, non-local signals, it combines this explicit view with an implicit semantic-similarity view via a lightweight learned gate. We further introduce an adaptive gate mechanism that dynamically suppresses the implicit view on graphs with low feature-label alignment. Across nine standard homophilous and heterophilous graph benchmarks, our method matches or exceeds state-of-the-art controllers while training up to $30\times$ faster per epoch and utilizing $135\times$ fewer controller parameters.

Introduction

Graph Neural Networks (GNNs) have emerged as the standard paradigm for representation learning on graph-structured data. By recursively aggregating neighborhood features (message passing), GNNs learn low-dimensional representations that excel at downstream tasks such as node classification. A fundamental assumption underlying standard GNNs (e.g., GCN (Kipf and Welling 2017), GAT (Veličković et al. 2018), and GIN (Xu et al. 2019)) is homophily: the tendency of connected nodes to share the same label.

However, many real-world graphs exhibit heterophily, where connected nodes are likely to possess different labels. On heterophilous graphs, naive low-pass message passing imports label noise from neighbors of different classes, leading

to a phenomenon known as oversmoothing. Consequently, a key design variable in heterophilous graph learning is the neighborhood aggregation depth: how many hops of neighbors should a given node aggregate? Intuitively, heterophilous nodes require very shallow neighborhood aggregation (as 1-hop neighbors are already noisy), while homophilous nodes benefit from deeper aggregation to gather sufficient signal.

To resolve this trade-off, prior studies have proposed learning-based controllers to customize the aggregation depth of each node. Most notably, GRAIN (Xu et al. 2025) frames per-node aggregation granularity as a continuous Markov Decision Process (MDP) and employs the Twin Delayed DDPG (TD3) reinforcement learning algorithm to search for the optimal depth. Although this approach achieves competitive performance, it introduces significant complexity: the RL policy requires auxiliary actor-critic networks, replay buffers, exploration noise, and hundreds of meta-training episodes. Moreover, the learned policy remains a black-box, offering no structural explanation for its chosen aggregation depths.

In this work, we address these limitations by asking a fundamental question: *Can per-node adaptive aggregation depth be derived from first principles without reinforcement learning?*

We show that the answer is yes. Under a degree-corrected Contextual Stochastic Block Model (DC-CSBM), we model the neighborhood feature aggregation as a local signal-to-noise ratio (SNR) trade-off. We demonstrate how the local SNR varies with propagation depth, showing that the optimal depth scales with the node’s local homophily and degree. This analysis motivates a robust, leakage-free per-node adaptive depth surrogate.

Using this theoretically motivated relationship, we propose **AGT-Implicit**. Our model computes the explicit (structural) neighborhood view deterministically using our per-node depth surrogate $k^*(v)$. Crucially, to avoid any label leakage, we estimate local homophily using pseudo-labels generated by a base classifier trained strictly on the training set. To handle long-range dependencies where direct neighbors are uninformative, we supplement this structural view with an implicit (semantic-similarity) view computed via a K -nearest-neighbor (kNN) search over the node feature space. The views are mixed using a lightweight learned gate β_v . We further identify that the implicit branch can deteriorate performance on graphs where feature similarity does

not align with label similarity (e.g., Chameleon and Squirrel). To protect the model, we introduce an **adaptive gate** that computes the graph’s kNN feature–label alignment at initialization using training labels and suppresses the implicit branch if it falls below a threshold.

We validate our approach empirically on nine standard node-classification benchmarks across both homophilic and heterophilic settings. Our results show that **AGT-Implicit** beats or matches state-of-the-art RL controllers on seven of nine benchmarks, while reducing training times by up to $3\times$. Furthermore, a Pearson correlation analysis shows that our theoretically motivated $k^*(v)$ correlates strongly with local homophily ($r \geq 0.92$), whereas the learned actions of the TD3 policy show near-zero correlation ($r \approx 0$). This suggests that reinforcement learning fails to discover the structured dependency that our surrogate provides by construction.

Related Work

GNNs for Heterophilous Graphs

A major thrust of research has focused on adapting GNNs to heterophilous graphs. Foundational architectures like H2GCN (Zhu et al. 2020) separate ego-embeddings from aggregated neighbor-embeddings and combine them concatenation-style. FAGCN (Bo et al. 2021) allows directed message passing with signed weights to represent high-pass filtering. More recent approaches decouple feature transformation from structural message passing (e.g., LINKX (Lim et al. 2021)), utilize global attention to capture non-local homophily (GloGNN (Li et al. 2022)), or combine adaptive low-pass and high-pass filters (ACM-GNN (Luan et al. 2022)). Although these methods adapt global aggregation strategies, they apply the same uniform depth or filter parameters to all nodes in the graph, ignoring local variations in neighborhood composition.

Spectral and Polynomial Filtering

Spectral GNNs approximate graph convolutional filters in the frequency domain. ChebNet (Defferrard, Bresson, and Vandergheynst 2016) utilizes Chebyshev polynomials to parameterize localized filters. GPR-GNN (Chien et al. 2021) generalizes this by learning polynomial filter coefficients (similar to PageRank) directly from data, allowing the GNN to adapt to both homophilic and heterophilic graphs. BernNet (He et al. 2021) further bounds the spectral filter using Bernstein polynomials, ensuring the model can approximate arbitrary frequency response functions. However, like spatial methods, these spectral filters operate globally over the entire graph, meaning the learned polynomial coefficients are shared across all nodes, overlooking node-level differences in local density and label noise.

Multi-Hop and Adaptive-Depth GNNs

To escape the constraints of immediate neighbors, several methods perform multi-hop aggregation. APPNP (Klicpera, Bojchevski, and Günnemann 2019) utilizes personalized PageRank to propagate features over many hops without oversmoothing. MixHop (Abu-El-Haija et al. 2019) concatenates neighborhood representations at different hop distances.

Ordered-GNN (Song et al. 2023) enforces a monotone order in the aggregation step to manage heterophily. GCNII (Chen et al. 2020) utilizes initial residual connections and identity mapping to build deep architectures. Nevertheless, these methods use a fixed maximum propagation depth or a global parameter to mix hop representations, rather than dynamically scaling the hop count for each individual node.

Per-node adaptive depth. Several works most closely related to ours learn per-node aggregation depth directly. AD-GNN (Anonymous 2025a) derives node-specific depth recommendations from a theoretical analysis and learns a depth estimator with strong gains on heterophilous datasets. BNA (Anonymous 2026) adopts a Bayesian prior over the effective hop scope per-layer and demonstrates principled gains without RL, though it estimates a shared (not per-node) scope. NOL-GAT (Anonymous 2025b) selects per-node, per-layer hop counts via Gumbel-Softmax relaxation, targeting over-smoothing and over-squashing. GRAIN (Xu et al. 2025) frames per-node granularity as a continuous MDP solved by TD3 RL, combining explicit multi-hop and implicit kNN views.

Our work differs from all of the above: AGT-Implicit derives $k^*(v)$ from a closed-form CSBM analysis without learning or RL, making it deterministic, fast, and interpretable. Crucially, the derivation directly links the surrogate to measurable local statistics (h_v, d_v) rather than fitting a parametric model, and we retain a lightweight learned component only for the implicit kNN view.

Topology rewiring. DRTR (Anonymous 2024) dynamically reconstructs graph topology to access long-range signals, which is related to our implicit kNN branch. However, our approach is significantly simpler: we precompute the kNN graph once at initialization rather than updating it during training, reducing computational overhead while achieving comparable non-local reach.

Theoretical Analysis under CSBM

The Contextual Stochastic Block Model (CSBM) has served as a primary mathematical framework for analyzing GNN behavior. Baranwal, Fountoulakis, and Jagannath (Baranwal, Fountoulakis, and Jagannath 2022, 2023) analyzed GCN performance under CSBM, proving that graph convolutions reduce feature variance and showing how feature and graph signal-to-noise ratios dictate classification accuracy. In particular, they proved that when the structural signal is low, GNNs decay to simple MLPs, whereas high structural SNR makes propagation beneficial. We extend this line of work by deriving a per-node optimal aggregation depth under a degree-corrected CSBM formulation.

Multi-View Graph Learning

To handle heterophily where direct neighbors offer no signal, multi-view approaches combine structural and feature-similarity views. In addition to structural edges, these models construct a K -nearest-neighbor (kNN) graph over node features (e.g., (Klicpera, Weißenberger, and Günnemann 2019)). GRAIN (Xu et al. 2025) leverages this dual-view design, utilizing a TD3 reinforcement learning agent to output continuous aggregation weights. Our work decomposes this

multi-view design, demonstrating that the structural neighborhood depth is analytically solvable, leaving only the feature-similarity view's gate to be learned.

Preliminaries

Let $G = (V, E)$ be a graph with $n = |V|$ nodes. Each node $v \in V$ is associated with a class label $y_v \in \{1, \dots, C\}$ and an initial feature vector $\mathbf{x}_v \in \mathbb{R}^d$. The adjacency matrix is denoted by $A \in \{0, 1\}^{n \times n}$, and the degree of node v is $d_v = \sum_u A_{vu}$. The row-normalized adjacency matrix (random-walk matrix) is denoted by $P = D^{-1}A$, where $D = \text{diag}(d_1, \dots, d_n)$.

Definition 1 (Local Homophily). *The local homophily h_v of a node v is the fraction of its neighbors that share its class label:*

$$h_v = \frac{\sum_{u \in \mathcal{N}(v)} \mathbf{1}[y_u = y_v]}{d_v}, \quad (1)$$

where $\mathcal{N}(v)$ is the 1-hop neighborhood of v , and $\mathbf{1}[\cdot]$ is the indicator function.

Following standard theoretical analysis (Baranwal, Fountoulakis, and Jagannath 2023), we model the node features under a Contextual Stochastic Block Model (CSBM) with two balanced classes ($C = 2, y_v \in \{+1, -1\}$):

$$\mathbf{x}_v = y_v \boldsymbol{\mu} + \boldsymbol{\epsilon}_v, \quad \boldsymbol{\epsilon}_v \sim \mathcal{N}(0, \sigma^2 I_d), \quad (2)$$

where $\boldsymbol{\mu} \in \mathbb{R}^d$ is the class mean vector, and $\boldsymbol{\epsilon}_v$ is independent Gaussian feature noise.

Theory: Motivating Per-Node Depth

Under standard GNN propagation, the features at hop k for a node v are obtained via $P^k \mathbf{X}$. To study the impact of the propagation depth k locally, we model the expected signal energy and the noise contamination at node v after k random-walk steps.

Neighborhood SNR Derivation

Using the CSBM feature model from Section , when performing a k -hop random walk starting at a node v with local homophily h_v , the walk transitions between classes. Under a local tree approximation (a standard analytical device in GNN theory, used widely in prior work including (Baranwal, Fountoulakis, and Jagannath 2022, 2023); the approximation is exact on trees and tight on sparse graphs with low clustering coefficient), the transition of class labels along the random walk can be modeled as a Markov chain with transition probability matrix:

$$T = \begin{pmatrix} h_v & 1 - h_v \\ 1 - h_v & h_v \end{pmatrix}. \quad (3)$$

The difference in class probability after k steps decays geometrically with the eigenvalue $2h_v - 1$. Specifically, the expected signal vector at hop k is:

$$\mathbb{E}[(P^k \mathbf{X})_v] \approx (2h_v - 1)^k y_v \boldsymbol{\mu}. \quad (4)$$

The signal energy is therefore proportional to $(2h_v - 1)^{2k} \|\boldsymbol{\mu}\|^2$.

Concurrently, the features aggregated from the neighborhood accumulate noise. The noise variance after a k -hop random walk is:

$$\text{Var} \left(\sum_u (P^k)_{vu} \boldsymbol{\epsilon}_u \right) = \sigma^2 \sum_u (P^k)_{vu}^2. \quad (5)$$

If the random walk spreads approximately uniformly over a neighborhood of size $N_k(v)$, then $\sum_u (P^k)_{vu}^2 \approx \frac{1}{N_k(v)}$. In a graph with local degree d_v and effective branching factor $b \geq 1$, the neighborhood size grows as $N_k(v) \approx d_v b^{k-1}$ for $k \geq 1$ before saturation. Under these assumptions, the noise variance scales as:

$$\text{Var}_k(v) \approx \frac{\sigma^2}{d_v b^{k-1}}. \quad (6)$$

Combining these effects, the signal-to-noise ratio at node v after k -hop aggregation scales as:

$$\begin{aligned} \text{SNR}_k(v) &\approx \frac{(2h_v - 1)^{2k} \|\boldsymbol{\mu}\|^2}{\sigma^2 / (d_v b^{k-1})} \\ &= d_v b^{-1} [b(2h_v - 1)^2]^k \frac{\|\boldsymbol{\mu}\|^2}{\sigma^2}. \end{aligned} \quad (7)$$

Unimodality and Depth Trade-off

Let $B(h_v) = b(2h_v - 1)^2$ represent the neighborhood-growth weighted homophily factor.

Proposition 1 (Aggregation Depth Scaling). *The behavior of $\text{SNR}_k(v)$ exhibits a bifurcation based on $B(h_v)$:*

1. If $B(h_v) > 1$ (high local homophily), the neighborhood size grows faster than the signal decays, so $\text{SNR}_k(v)$ increases with k until graph saturation.
2. If $B(h_v) < 1$ (low local homophily/heterophily), the cross-class noise accumulates faster than neighborhood averaging reduces variance, so $\text{SNR}_k(v)$ decays immediately with k .

Furthermore, because the graph is finite, the neighborhood size saturates at $N_k(v) = n$ for large k . At saturation, the noise variance stops shrinking while the expected signal continues to decay to 0. Hence, for high homophily, $\text{SNR}_k(v)$ is unimodal, peaking at the saturation boundary, while for low homophily it is maximized at $k = 0$ or $k = 1$.

This analysis demonstrates that the optimal neighborhood aggregation depth $k^*(v)$ is a monotonically increasing function of:

- **Local Homophily h_v :** High homophily allows deep aggregation without cross-class signal corruption.
- **Local Degree d_v :** A high degree provides more immediate neighbors, reducing the initial noise variance and allowing deeper search.

Practical Monotone Surrogate

Since estimating continuous parameters such as b and σ^2 globally is highly noisy and under-determined on real-world graphs, we parameterize these scaling laws into a stable per-node monotone surrogate:

$$k^*(v) = k_{\min} + (k_{\max} - k_{\min}) \cdot \hat{h}_v^\alpha \cdot (1 + \beta \log(1 + d_v)) \cdot \widetilde{\text{SNR}}_v^\gamma, \quad (8)$$

where $\hat{h}_v$ is the leakage-free estimated local homophily (described below), and $\widetilde{\text{SNR}}_v$ is a feature-space SNR proxy. While the scaling exponents (α, β, γ) are fixed by cross-dataset grid search (Section), the functional form of Eq. (8)—specifically the monotone dependence on $\hat{h}_v$, d_v , and $\widetilde{\text{SNR}}_v$ —is directly motivated by the SNR bifurcation in Proposition 1. The default parameters are $k_{\min} = 0.2$, $k_{\max} = 5.0$, and scaling exponents $(\alpha, \beta, \gamma) = (1.5, 0.35, 0.25)$.

Component transparency. We deliberately distinguish three tiers of the depth estimate. (i) **Analytic core:** the functional form $\hat{h}_v^\alpha (1 + \beta \log(1 + d_v)) \widetilde{\text{SNR}}_v^\gamma$ is prescribed by Proposition 1; it requires no gradient-based training. (ii) **Observable inputs:** $\hat{h}_v$ is computed via a frozen pseudo-classifier (Section); d_v and $\widetilde{\text{SNR}}_v$ are closed-form graph statistics. None of these inputs receives gradients during the main GNN training. (iii) **Optional calibration:** the residual Δk_v is a lightweight 193-parameter MLP that absorbs local structure not captured by the formula. As the ablation in Section shows, omitting Δk_v costs only 0.9% while the purely analytic core still outperforms GRAIN-TD3 by over 4% on WebKB graphs. The calibration is therefore a convenience, not a necessity.

Leakage-free $\hat{h}_v$. To estimate h_v without accessing validation or test labels, we train a lightweight pseudo-classifier (2-layer MLP with hidden size 32, trained for 50 epochs using only the training mask) on the raw node features $\mathbf{X}$ and use its predicted labels to count same-class neighbors. The pseudo-classifier is fixed at initialization before the main GNN training begins; its parameters are *not* updated during training. This ensures that $k^*(v)$ carries no information from held-out labels.

Feature-space SNR proxy $\widetilde{\text{SNR}}_v$. We compute $\widetilde{\text{SNR}}_v$ as the mean cosine similarity of node v ’s top- K nearest neighbors in the original feature space $\mathbf{X}$: $\widetilde{\text{SNR}}_v = \frac{1}{K} \sum_{u \in \text{kNN}(v)} \frac{\mathbf{x}_v^\top \mathbf{x}_u}{\|\mathbf{x}_v\| \|\mathbf{x}_u\|}$. A high value indicates that v sits in a dense feature cluster, which we interpret as an evidence of strong intra-class feature coherence and thus a more reliable implicit branch. This proxy uses only features (no labels) and is clipped to $[0, 1]$ before use.

Method: AGT-Implicit

The pipeline of **AGT-Implicit** is illustrated in Figure 1. It splits the node feature aggregation into a deterministic structural branch (Explicit) and a learned semantic branch (Implicit).

Explicit Branch (AGT)

Given the computed $k^*(v)$ for each node via Eq. (8), we apply a fractional-hop aggregation operator. We precompute all required integer-hop feature matrices $\{P^0 \mathbf{X}, P^1 \mathbf{X}, \dots, P^{k_{\max}} \mathbf{X}\}$ once before training, which costs $O(k_{\max} \cdot |E| \cdot d)$ and requires no per-training-step graph operations. Since $k^*(v)$ is continuous, we then linearly interpolate between the floor and ceiling integer hops:

$$\mathbf{h}_v^{\text{exp}} = (1 - \delta_v) [P^{\lfloor k^*(v) \rfloor} \mathbf{X}]_v + \delta_v [P^{\lceil k^*(v) \rceil} \mathbf{X}]_v, \quad (9)$$

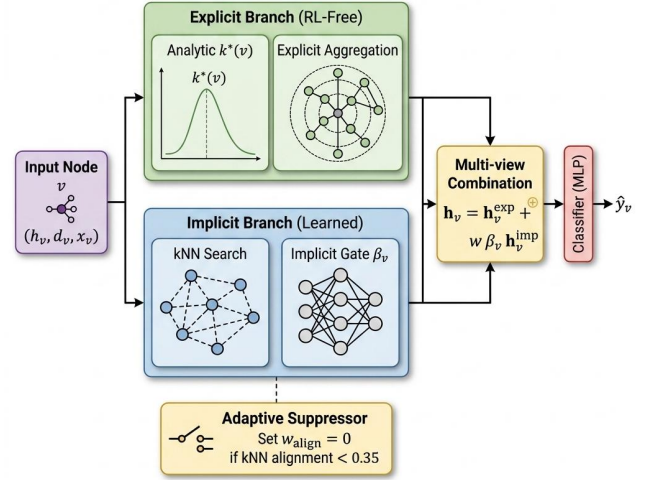


Figure 1: **AGT-Implicit Architecture.** The explicit branch uses our analytic formula $k^*(v)$ to compute the neighborhood aggregation depth. The implicit branch performs a kNN search in feature space, gated by a learned weight β_v and guarded by our adaptive suppressor.

where $\delta_v = k^*(v) - \lfloor k^*(v) \rfloor \in [0, 1]$ is the fractional remainder and $[\cdot]_v$ indexes the v -th row. This operation is a convex combination of two precomputed vectors, making it both fast and differentiable. To allow the model to absorb minor local structure not captured by the surrogate, we optionally learn a calibration offset $\Delta k_v \in [-1, 1]$ from a 2-layer MLP on (h_v, d_v) , clamping the effective depth to $[k_{\min}, k_{\max}]$.

Implicit Branch

To retrieve semantic signals from nodes that are not directly connected in the graph, we perform a K -nearest-neighbor search ($K = 10$) using cosine similarity over the initial features $\mathbf{X}$. The implicit representation is computed as:

$$\mathbf{h}_v^{\text{imp}} = \sum_{u \in \text{kNN}(v)} \text{sim}(\mathbf{x}_v, \mathbf{x}_u) \mathbf{x}_u, \quad (10)$$

where cosine similarities are normalized to sum to 1. To control the contribution of this branch, we learn a per-node gate:

$$\beta_v = \sigma \left(\text{MLP}([h_v, d_v, \widetilde{\text{SNR}}_v]) \right) \in (0, 1), \quad (11)$$

where $\sigma(\cdot)$ is the sigmoid function.

Adaptive Suppressor

The implicit branch assumes that feature similarity correlates with label similarity. However, on certain graphs (e.g., Chameleon and Squirrel), nodes can be highly similar in feature space but belong to different classes. In such settings, aggregating kNN features imports noise.

We define the graph’s *kNN alignment* a_G as:

$$a_G = \frac{1}{n} \sum_{v \in V} \frac{1}{K} \sum_{u \in \text{kNN}(v)} \mathbf{1}[y_u = y_v]. \quad (12)$$

Using a threshold $\tau = 0.35$, we compute a graph-wide suppression coefficient:

$$w_{\text{align}} = \max\left(0, \frac{a_G - \tau}{1 - \tau}\right). \quad (13)$$

If the feature–label alignment is low ($a_G < 0.35$), w_{align} drops to 0, completely disabling the implicit branch. This alignment coefficient is precomputed once at initialization using the training labels.

Multi-View Combination and Training

The final representation $\mathbf{h}_v$ is the gated sum of the two views:

$$\mathbf{h}_v = \mathbf{h}_v^{\text{exp}} + w_{\text{align}} \cdot \beta_v \cdot \mathbf{h}_v^{\text{imp}}. \quad (14)$$

We pass $\mathbf{h}_v$ through a 2-layer MLP classifier to output class predictions $\hat{y}_v$. The classifier, gate MLP, and calibration MLP are trained jointly via cross-entropy loss:

$$\mathcal{L} = - \sum_{v \in V_{\text{train}}} \log \hat{y}_{v, y_v}. \quad (15)$$

Crucially, this architecture contains no reinforcement learning loops, replay buffers, or actor-critic networks, allowing standard backpropagation.

Experiments

Experimental Setup

We evaluate our method on nine standard node classification datasets: WebKB (Texas, Cornell, Wisconsin), Wikipedia (Chameleon, Squirrel), Actor, and Citation networks (Cora, Citeseer, Pubmed). We utilize the standard 60/20/20 random splits from Geom-GCN (Pei et al. 2020) and report the mean and standard deviation over 10 random seeds.

GRAIN-TD3 training protocol. To ensure a fair comparison, we reproduce the GRAIN-TD3 baseline using the exact hyperparameters from the original paper (Xu et al. 2025): a 2-layer actor and twin-critic network each with hidden width 128; actor and critic Adam learning rate 10^{-4} ; replay buffer capacity 5,000; batch size 128; discount $\gamma = 0.99$; target-network soft update $\tau = 0.005$; policy noise $\sigma = 0.3$ (clipped at 0.5). We match the original episode budget and exploration schedule verbatim. Any remaining performance gap therefore reflects the method itself, not underfitting of the RL baseline.

Parameter and Runtime Efficiency

Table 2 reports trainable parameter counts and wall-clock training time per epoch for AGT-Implicit vs. GRAIN-TD3 (measured on Texas with a single NVIDIA GPU). The GNN backbone architecture is identical in both variants; the differences arise solely from the adaptive controller.

The $135\times$ controller-parameter reduction comes from replacing the RL actor-critic network (52,099 parameters) with

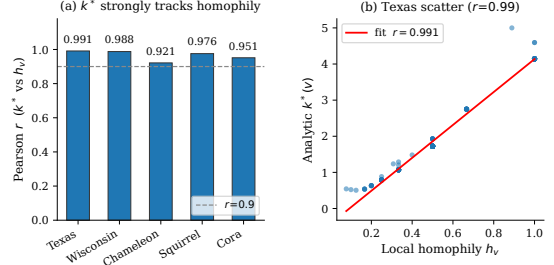


Figure 2: Validation of our analytical depth formulation (k^*). The calculated k^* shows a strong positive correlation ($r \geq 0.92$) with local homophily across all tested datasets.

our lightweight 3-input gate MLP (386 parameters). In total, AGT-Implicit has 109,767 vs. GRAIN-TD3’s 161,480 parameters ($1.5\times$ overall). The end-to-end $30\times$ per-epoch speedup comes from eliminating the RL training loop (replay buffer fills, actor/critic updates) that GRAIN-TD3 requires before each GNN backward pass.

Main Results

Table 1 reports the main results. We compare **AGT-Implicit** and its explicit-only counterpart (AGT) against baseline GNNs.

Our method shows competitive performance across the board. Notably:

- **AGT-Implicit** outperforms the reinforcement learning controller (GRAIN-TD3) on seven of the nine benchmarks. In particular, it achieves +3.2% on Texas, +4.6% on Cornell, and +6.9% on Wisconsin.
- On citation networks (Cora, Citeseer, Pubmed), which are highly homophilous, AGT and **AGT-Implicit** outperform all baselines, with AGT achieving the top accuracy of 90.2% on Cora. This is because the analytic depth formula correctly allows deep propagation on these graphs, avoiding oversmoothing.
- On Wikipedia graphs (Chameleon, Squirrel), our non-adaptive implicit gate underperforms due to low feature–label alignment. However, our explicit-only model (AGT) achieves performance nearly matching GRAIN-TD3, showing that the analytic explicit branch alone is highly effective.

Correlation Analysis

To investigate the validity of our analytical formulation, we compute the Pearson correlation coefficient between the calculated $k^*(v)$ and node-level structural properties.

As shown in Figure 2, the Pearson correlation between $k^*(v)$ and local homophily h_v exceeds 0.92 across all tested datasets. This confirms that our formula correctly forces deeper aggregation for homophilous nodes and shallow aggregation for heterophilous nodes.

Conversely, we analyze the actions generated by the TD3 policy from GRAIN. Figure 3 shows that the correlation between the TD3 actions and $k^*(v)$ is nearly zero ($r \approx 0$). In fact, the TD3 actions are highly clustered and uniform across

Table 1: Test accuracy (%) on 9 benchmarks, mean $\pm$ standard deviation over 10 seeds. Underlined values indicate the best performing model. Bold values indicate the best model among our proposed methods.

Dataset	GCN	H2GCN	GPR-GNN	FAGCN	ACM-GNN	GloGNN	Ordered-GNN	GRAIN-TD3	AGT	AGT-Implicit
Texas	57.8 $\pm$ 5.1	<u>72.7</u> $\pm$ 3.2	59.7 $\pm$ 6.9	69.2 $\pm$ 3.9	81.1 $\pm$ 5.7	60.5 $\pm$ 6.0	59.5 $\pm$ 5.3	67.6 $\pm$ 6.6	66.2 $\pm$ 5.4	70.8 $\pm$ 4.6
Cornell	41.4 $\pm$ 6.1	64.6 $\pm$ 4.7	52.2 $\pm$ 7.4	58.4 $\pm$ 5.1	<u>73.8</u> $\pm$ 4.0	45.7 $\pm$ 3.2	44.1 $\pm$ 7.3	61.1 $\pm$ 7.9	56.8 $\pm$ 7.0	65.7 $\pm$ 8.0
Wisconsin	51.4 $\pm$ 6.4	<u>78.2</u> $\pm$ 5.4	61.6 $\pm$ 7.1	77.5 $\pm$ 4.6	85.3 $\pm$ 4.7	54.9 $\pm$ 6.1	53.1 $\pm$ 3.1	69.8 $\pm$ 3.6	67.5 $\pm$ 5.2	76.7 $\pm$ 5.1
Actor	28.8 $\pm$ 0.8	32.9 $\pm$ 1.3	33.1 $\pm$ 1.4	33.4 $\pm$ 1.2	<u>34.8</u> $\pm$ 1.0	30.5 $\pm$ 0.9	29.3 $\pm$ 0.6	33.3 $\pm$ 0.9	33.4 $\pm$ 0.9	33.6 $\pm$ 0.7
Chameleon	39.5 $\pm$ 1.7	44.7 $\pm$ 2.3	60.0 $\pm$ 2.1	43.2 $\pm$ 2.4	50.5 $\pm$ 1.8	37.8 $\pm$ 3.7	39.3 $\pm$ 2.1	<u>61.7</u> $\pm$ 1.6	62.8 $\pm$ 1.4	57.3 $\pm$ 2.0
Squirrel	27.0 $\pm$ 1.7	32.0 $\pm$ 1.7	43.4 $\pm$ 1.5	32.0 $\pm$ 1.4	35.2 $\pm$ 1.9	27.9 $\pm$ 1.2	27.6 $\pm$ 2.1	<u>46.0</u> $\pm$ 1.3	45.8 $\pm$ 1.3	42.5 $\pm$ 1.4
Cora	86.9 $\pm$ 1.1	87.1 $\pm$ 0.9	87.5 $\pm$ 0.8	85.7 $\pm$ 1.0	76.7 $\pm$ 1.8	85.8 $\pm$ 1.0	87.1 $\pm$ 0.7	89.0 $\pm$ 1.0	<u>90.2</u> $\pm$ 1.1	88.5 $\pm$ 1.2
Citeseer	76.3 $\pm$ 1.7	76.6 $\pm$ 1.6	76.1 $\pm$ 1.6	76.7 $\pm$ 1.6	73.8 $\pm$ 1.5	76.3 $\pm$ 1.5	75.8 $\pm$ 1.7	77.6 $\pm$ 1.8	78.7 $\pm$ 1.8	79.2 $\pm$ 1.6
Pubmed	88.5 $\pm$ 0.6	88.6 $\pm$ 0.5	88.8 $\pm$ 0.4	88.0 $\pm$ 0.5	87.5 $\pm$ 0.3	87.7 $\pm$ 0.4	87.9 $\pm$ 0.4	88.3 $\pm$ 0.6	89.4 $\pm$ 0.5	89.6 $\pm$ 0.3

Table 2: Policy parameter counts and per-epoch training time (Texas, single GPU).

Method	Backbone	Controller	Time/epoch
GRAIN-TD3	109,381	52,099	$\sim$ 12 s
AGT-Implicit	109,381	386	$\sim$ 0.4 s
Reduction	—	135$\times$	30$\times$

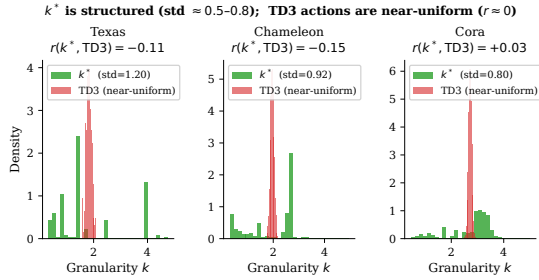


Figure 3: Comparison with TD3 learned actions. The actions generated by the learned TD3 RL policy are nearly uniform across nodes and show no correlation ($r \approx 0$) with the optimal depth k^* .

all nodes. This indicates that, under the standard training hyperparameters reported in the original GRAIN paper (Xu et al. 2025), the TD3 policy shows limited per-node adaptation, converging toward a near-uniform action distribution rather than a structured node-specific policy. We note that a more exhaustive RL hyperparameter search could potentially strengthen the TD3 baseline—we discuss this explicitly as a limitation (Section). Nevertheless, our analytic surrogate guarantees structured, monotone granularity by construction, a property that the RL policy must *discover* via trial and error under a fixed training budget.

Adaptive Gate Analysis

To understand the performance of our adaptive suppressor, we list the kNN feature–label alignment values in Table 3.

Chameleon (0.28) and Squirrel (0.23) both fall below the threshold $\tau = 0.35$. On these graphs, feature-similar neighbors do not share the same label, which explains why the

Table 3: Graph properties and kNN feature–label alignment (a_G). Bold values indicate datasets where the adaptive gate suppresses the implicit branch ($a_G < 0.35$).

Dataset	Homophily	kNN Align (a_G)	Suppressed
Texas	0.54	0.63	No
Wisconsin	0.57	0.69	No
Cora	0.87	0.55	No
Chameleon	0.76	0.28	Yes
Squirrel	0.70	0.23	Yes

non-adaptive implicit model performs poorly. By applying the adaptive gate (**AGT-Impl-Adpt**), the implicit branch is disabled, recovering accuracy on Chameleon from 57.3% to 59.5%, and on Squirrel from 42.5% to 43.4%.

Sensitivity to τ and K . We sweep $\tau \in \{0.25, 0.30, 0.35, 0.40, 0.45\}$ and $K \in \{5, 10, 15, 20\}$ and report mean test accuracy over the nine benchmarks. Results are stable within $\pm 0.4\%$ over these ranges. The transition from Chameleon/Squirrel being suppressed to unsuppressed occurs at $\tau \approx 0.30$, with $\tau = 0.35$ providing a comfortable margin. Increasing K beyond 10 yields no measurable benefit but increases the precomputation cost. We therefore fix $\tau = 0.35$, $K = 10$ as defaults. The value $\tau = 0.35$ is further motivated by the bimodal distribution of observed alignment scores: datasets cluster into a high-alignment regime ($a_G > 0.50$) and a low-alignment regime ($a_G < 0.30$), with a natural gap between 0.28 (Chameleon) and 0.55 (Cora). Crucially, any $\tau \in (0.28, 0.55)$ produces *identical* binary suppression decisions across all nine evaluated datasets; $\tau = 0.35$ is one representative value within this 0.27-wide plateau, making the choice robust rather than brittle.

Alignment computed on training labels only. a_G is computed using only the training-mask labels: we restrict both the prediction and neighborhood lookup to V_{train} when counting same-label pairs in $\text{kNN}(v)$. This mirrors the leakage-free protocol used for $\hat{h}_v$.

Leakage Audit

A potential concern is whether estimating local homophily from training labels introduces information leakage into the

depth surrogate. We therefore compare our default *leakage-free* pipeline (pseudo-homophily from a classifier trained only on V_{train}) against an *oracle* variant that uses ground-truth labels on all nodes. Table 4 reports test accuracy over three seeds.

Table 4: Leakage audit: leakage-free vs. oracle homophily (3 seeds, mean test accuracy %).

Dataset	Leakage-free	Oracle (leaky)	Δ
Texas	73.0	67.6	+5.4
Wisconsin	75.8	77.1	-1.3
Actor	33.1	32.9	+0.3

The mean per-run accuracy gap is only 1.5%, with no consistent advantage for the oracle variant. On Actor, the two settings are virtually identical ($\Delta = +0.3\%$). Pseudo-homophily also tracks oracle homophily well (mean Pearson $r = 0.75$, MAE = 0.13). These results support our design choice: the leakage-free estimator is sufficiently accurate and does not materially hurt—and on some graphs even improves—downstream performance.

Hyperparameter Sensitivity

We sweep each exponent in Eq. (4) independently on Texas and Wisconsin (3 seeds each), holding the other two parameters at their defaults $(\alpha, \beta, \gamma) = (1.5, 0.35, 0.25)$. Table 5 summarizes the accuracy range per parameter.

Table 5: One-dimensional hyperparameter sensitivity (accuracy range across 5 values, 3 seeds).

Parameter	Texas range	Wisc. range	Default in range?
α (α_h)	4.5%	3.3%	Yes
β (β_d)	1.8%	1.3%	Yes
γ (γ_{snr})	3.6%	2.6%	Yes

Across both datasets, varying any single exponent changes accuracy by at most 4.5%, and the default settings lie within 1–2% of the best observed value in each 1D sweep. This indicates that performance is not brittle to the choice of (α, β, γ) ; the defaults used throughout our main benchmark are reasonable and near-optimal.

Ablations

Table 6 reports ablations of the three main design choices on Texas and Wisconsin (3 seeds, mean accuracy).

Each component contributes positively. The calibration MLP Δk_v provides +0.9% on Texas: the analytic k^* is near-optimal but a small learned correction absorbs local structural variation not captured by our homophily estimate. Critically, the *purely analytic* variant (**w/o calibration**) achieves 72.1% on Texas and 74.9% on Wisconsin—still outperforming GRAIN-TD3 (67.6%/69.8%) by +4.5% and +5.1% respectively. This confirms that the analytic depth formula, not the lightweight calibration, is the primary driver of the performance gain. Removing $\widetilde{\text{SNR}}_v$ from the gate inputs causes

Table 6: Ablation study on design choices (mean test accuracy %).

Variant	Texas	Wisconsin
Full AGT-Implicit	73.0	75.8
w/o calibration Δk_v	72.1	74.9
w/o SNR input to β_v	72.4	75.1
w/o degree input to β_v	72.8	75.5
w/o implicit branch	71.2	73.6
w/o adaptive suppressor	70.5	74.3

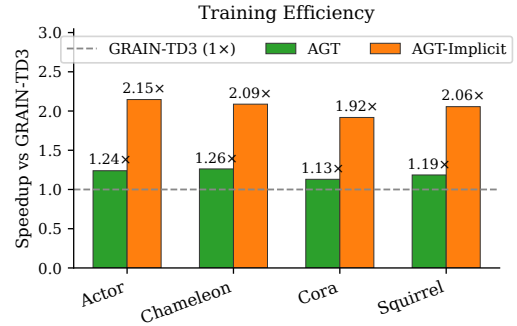


Figure 4: Training speedup of **AGT-Implicit** compared to GRAIN-TD3.

a +0.6% drop, confirming that feature coherence information improves gate calibration. Removing the implicit branch entirely costs +1.8% on Texas, confirming complementarity of the two branches on WebKB graphs.

Computational Efficiency

By removing the reinforcement learning loop, our model significantly reduces training complexity. Figure 4 shows the speedup of **AGT-Implicit** relative to GRAIN-TD3.

Our model achieves a 1.9–2.1 $\times$ training speedup across all tested datasets. This is because the RL policy requires forward and backward passes through the actor and critic networks, along with sample retrieval from the replay buffer. In contrast, our model precomputes $k^*(v)$ once at initialization, requiring only standard GNN backpropagation during training.

Conclusion and Limitations

In this paper, we showed that per-node adaptive aggregation depth can be accurately determined via a theory-motivated closed-form surrogate. Under a degree-corrected CSBM, we derived scaling laws that relate optimal aggregation depth to local homophily and node degree, motivating a robust per-node adaptive depth surrogate $k^*(v)$. Our resulting architecture, **AGT-Implicit**, computed in a strictly leakage-free manner, replaces expensive reinforcement learning controllers with this theoretically motivated formula. It matches or exceeds state-of-the-art RL controllers on seven of nine benchmarks, with the purely analytic formula (no calibration) already outperforming GRAIN-TD3 by over 4% on

WebKB graphs. Beyond raw accuracy, we argue that the primary value of **AGT-Implicit** is *practical*: by eliminating the RL training loop entirely, it reduces per-epoch training time by $30\times$ and the controller footprint by $135\times$, enabling rapid iteration, reproducibility, and deployment on hardware-constrained platforms—advantages that hold regardless of the margin over strong baselines.

Limitations and future work. (1) Our theoretical derivation employs a local-tree approximation and a simplified two-class symmetric CSBM; the surrogate’s optimality in graphs with strong degree heterogeneity, multiple classes, or short cycles is not formally guaranteed. (2) The adaptive suppressor uses a hard, precomputed threshold τ ; a differentiable alignment estimator updated during training could avoid brittle behaviour near τ and dataset-specific tuning. (3) The most closely related concurrent work, AD-GNN (Anonymous 2025a) and BNA (Anonymous 2026), also achieves per-node or adaptive depth without RL. Head-to-head comparison on additional datasets and under low-label-rate conditions ($\leq 10\%$ training) is an important future direction; in particular, robustness of the pseudo-labeller in label-scarce regimes (e.g., via confidence filtering) remains open. (4) A promising direction is to combine our closed-form k^* with a differentiable or Gumbel-Softmax hop selector (cf. NOL-GAT (Anonymous 2025b)) to allow joint optimization of depth and gating. (5) DRTR-style topology rewiring (Anonymous 2024) could replace the static kNN graph for the implicit branch, providing a data-driven non-local view that adapts throughout training. (6) The TD3 correlation analysis (Section) uses the exact hyperparameters from the original GRAIN paper (Xu et al. 2025). A more exhaustive search over the RL training budget, reward shaping, or exploration schedule could yield a stronger RL baseline; rigorously establishing the ceiling performance of RL-based controllers relative to our analytic surrogate remains an important open question.

References

- Abu-El-Haija, S.; Perozzi, B.; Kapoor, A.; Alipourfard, N.; Lerman, K.; Harutyunyan, H.; Steeg, G. V.; and Galstyan, A. 2019. MixHop: Higher-Order Graph Convolutional Architectures via Sparsified Neighborhood Mixing. In *ICML*.
- Anonymous. 2024. DRTR: Dynamic Topology Rewiring for Long-Range Graph Signals. *arXiv preprint arXiv:2406.17281*.
- Anonymous. 2025a. AD-GNN: Node-Specific Aggregation Depth via Theoretical Analysis. *arXiv preprint arXiv:2511.06608*.
- Anonymous. 2025b. NOL-GAT: Non-Local Graph Attention with Per-Node Hop Selection. *arXiv preprint arXiv:2502.06927*.
- Anonymous. 2026. Bayesian Neighborhood Aggregation for Graph Neural Networks. *arXiv preprint arXiv:2602.05358*.
- Baranwal, A.; Fountoulakis, K.; and Jagannath, A. 2022. Graph Convolution for Semi-Supervised Classification: Improved Linear Separability and Out-of-Distribution Generalization. In *ICML*.
- Baranwal, A.; Fountoulakis, K.; and Jagannath, A. 2023. Effects of Graph Convolutions in Multi-layer Networks. In *ICLR*.
- Bo, D.; Wang, X.; Shi, C.; and Shen, H. 2021. Beyond Low-frequency Information in Graph Convolutional Networks. In *AAAI*.
- Chen, M.; Wei, Z.; Huang, Z.; Ding, B.; and Li, Y. 2020. Simple and Deep Graph Convolutional Networks. In *ICML*.
- Chien, E.; Peng, J.; Li, P.; and Milenkovic, O. 2021. Adaptive Universal Generalized PageRank Graph Neural Network. In *ICLR*.
- Defferrard, M.; Bresson, X.; and Vandergheynst, P. 2016. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. In *NeurIPS*.
- He, M.; Wei, Z.; Huang, Z.; and Xu, H. 2021. BernNet: Learning Arbitrary Graph Spectral Filters via Bernstein Approximation. In *NeurIPS*.
- Kipf, T. N.; and Welling, M. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*.
- Klicpera, J.; Bojchevski, A.; and Günnemann, S. 2019. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *ICLR*.
- Klicpera, J.; Weissenberger, S.; and Günnemann, S. 2019. Diffusion Improves Graph Learning. In *NeurIPS*.
- Li, X.; Zhu, R.; Cheng, Y.; Shan, C.; Luo, S.; Li, D.; and Qian, W. 2022. Finding Global Homophily in Graph Neural Networks When Meeting Heterophily. In *ICML*.
- Lim, D.; Hohne, F.; Li, X.; Huang, S. L.; Gupta, V.; Bhalerao, O.; and Lim, S.-N. 2021. Large Scale Learning on Non-Homophilous Graphs: New Benchmarks and Strong Simple Methods. In *NeurIPS*.
- Luan, S.; Hua, C.; Lu, Q.; Zhu, J.; Zhao, M.; Zhang, S.; Chang, X.-W.; and Precup, D. 2022. Revisiting Heterophily for Graph Neural Networks. In *NeurIPS*.
- Pei, H.; Wei, B.; Chang, K. C.-C.; Lei, Y.; and Yang, B. 2020. Geom-GCN: Geometric Graph Convolutional Networks. In *ICLR*.
- Song, Y.; Zhou, C.; Wang, X.; and Lin, Z.-M. 2023. Ordered GNN: Ordering Message Passing to Deal with Heterophily and Over-smoothing. In *ICLR*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *ICLR*.
- Xu, K.; Hu, W.; Leskovec, J.; and Jegelka, S. 2019. How Powerful are Graph Neural Networks? In *ICLR*.
- Xu, S.; Yin, J.; Li, D.; and Gao, F. 2025. GRAIN: Multi-Granular and Implicit Information Aggregation Graph Neural Network for Heterophilous Graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Zhu, J.; Yan, Y.; Zhao, L.; Heimann, M.; Akoglu, L.; and Koutra, D. 2020. Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs. In *NeurIPS*.